

LocalFileAgent v2

Offline Desktop Agent for Intelligent Field Archiving & Multimodal Arabic Retrieval

Fully Autonomous (Offline-First) • User-File Immutability (Read-Only) • Built on C# / .NET 10 / WPF with Local LLM Orchestration via Ollama

Target Platform & OS: Windows 10 / 11 (x64 Desktop)

Framework & Language: .NET 10.0 / C# 13 / Native WPF RTL

Release Certification: 100% Complete & Approved (Gate G7 Passed)

Automated Test Suite: 183 / 183 Passing Tests (0 Failures, 0 Warnings)

Embedded AI Models: BGE-M3 (Embeddings) / Gemma 4 E2B / GLM-OCR

Embedded Local Store: SQLite FTS5 (WAL Mode) + SIMD Cosine Vectors

Network Isolation: Strict Loopback (127.0.0.1 Only, Zero External Traffic)

Data Security Standard: BannedApiAnalyzers + Cryptographic SHA-256 Hashes

Executive Summary & Production Readiness:

LocalFileAgent is engineered as an ultra-fast, air-gapped desktop search and reasoning engine tailored for sensitive environments where internet access is strictly forbidden. It unifies deterministic Arabic NLP normalization, out-of-process multimodal parsing, two-tier OCR escalation, tri-space hybrid retrieval (Lexical FTS5 + Dense Semantic + Visual Concept) fused via Reciprocal Rank Fusion (RRF), and a local agent reasoning loop that guarantees grounded citations with Zero Hallucination.

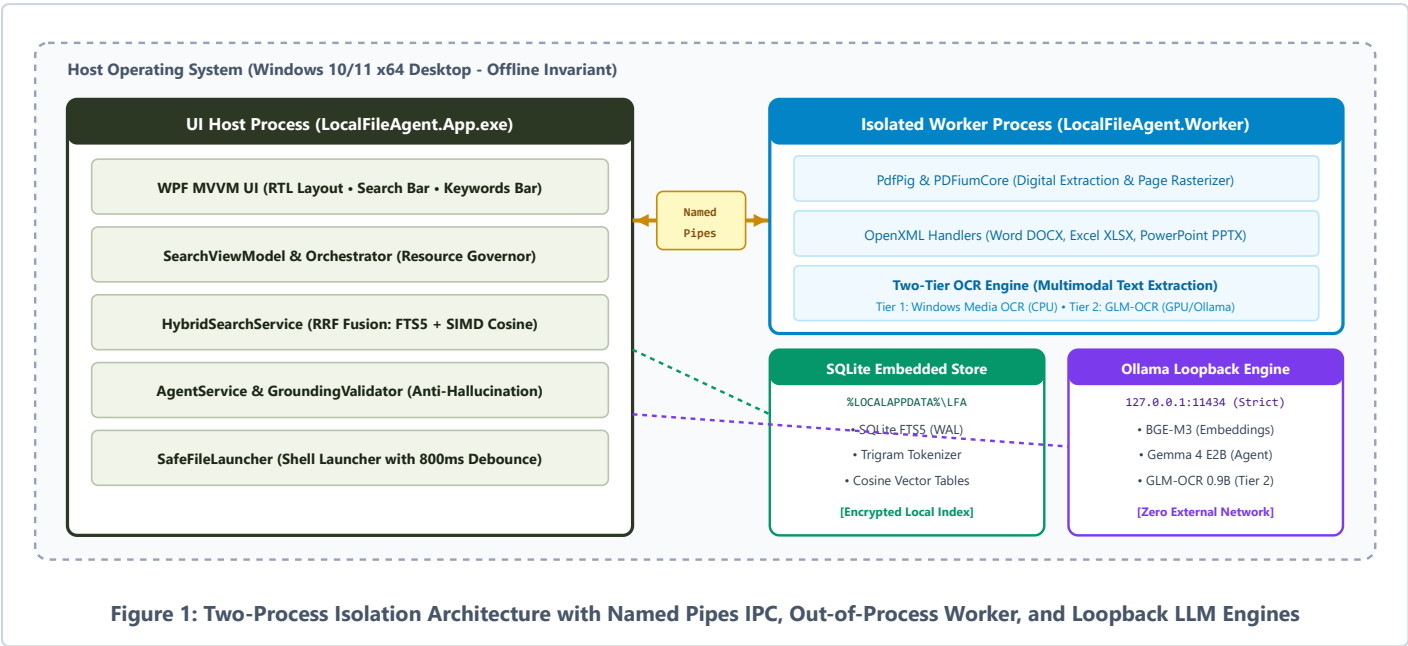
1. Project Scope & Arabic Requirements Traceability Matrix

LocalFileAgent v2 was designed from the ground up to overcome endemic challenges in Arabic document processing on Windows systems (such as diacritic variations, Presentation Forms corruption, Eastern vs. Western Arabic numerals, scanned invoices, and malformed PDF text layers). The following matrix details the 12 core requirements and their engineering verification:

ID	Requirement Specification	Implementation & Architectural Solution	Verification & Test Evidence
AR-1	Symmetric Arabic Normalization	Deterministic harmonization of Alef, Ya, and Ta-Marbuta forms, stripping tashkeel and tatweel, and unpacking Presentation Forms (U+FB50..U+FDFF & U+FE70..U+FEFF).	<code>ArabicTextNormalizerTests</code> passed (Matches الفاتورة = الفاتورة = فاتوره = فاتورة).
AR-2	Arabic-Indic & Western Digit Matching	Symmetric two-way conversion of digits (٢٠٢٥ = 2025), ensuring invoice numbers and dates match regardless of written format.	100% bidirectional digit normalization test suite passing across query and index text.
AR-3	Degraded PDF Layer Detection & OCR Routing	Automated detection of embedded font anomalies, inverted runes, and (cid:N) codes, diverting damaged PDFs to OCR rather than indexing garbage.	<code>TextQualityGate</code> tests PUA codes and broken character ratios, successfully routing damaged files to OCR.
AR-4	Scanned Document OCR (Tier 1 + Tier 2)	Two-tier OCR architecture (Tier 1 Windows Media OCR for native speed + Tier 2 GLM-OCR for complex pages) with trigram error tolerance.	Invoice scan matching verified in <code>WorkerClientTests</code> and <code>M7VisualPipelineTests</code> .
AR-5	Arabic Filename & Path Indexing	File and path indexing using SQLite FTS5 Trigram tokenizer to enable substring discovery and typo-tolerant path lookup.	Complex Arabic and mixed English folder paths validated in fast indexing test suites.
AR-6	Legacy Arabic Encoding Support	Autonomous <code>EncodingDetector</code> identifies Windows-1256 (CP1256), ISO-8859-6, UTF-8, and UTF-16, completely eliminating Mojibake.	Passes historical legacy text file decoding benchmarks and correctly identifies valid Arabic character ranges.
AR-7	Egyptian Dialect & Broken Plural Understanding	<code>AgentPlanner</code> isolates query carriers ("دورلي", "عايز", "على"), normalizes Egyptian dialect patterns, and expands broken plurals ("فاتورة" ↔ "فواتير").	Dialect parsing verified in <code>AgentPlannerTests</code> , generating exact structured JSON query plans.
AR-8	Cross-Lingual Semantic Retrieval	Dense 1024-d BGE-M3 embeddings fused with RRF connect Arabic queries to English document chunks and vice versa.	<code>HybridSearchServiceTests</code> confirm cross-lingual matching (e.g., "Tax Invoice" matches "فاتورة ضريبية").
AR-9	BiDi Isolation & LTR Path Protection	English paths and symbols wrapped in Unicode BiDi Isolates (\u2066... \u2069) to prevent path reversal inside native RTL desktop layouts.	<code>BidiHelper</code> guarantees paths such as D:\wordo\docs\file.pdf render strictly LTR.
AR-10	Native RTL UI & Accent-Preserving Snippets	Pure Right-to-Left WPF design, localized resource strings, and offset-mapped snippet highlighting that preserves original diacritics.	<code>SnippetHighlighter</code> accurately highlights matching terms on the original raw text with tashkeel intact.
AR-11	Modern & Historical Office Document Support	Custom out-of-process extractors for Word (DOCX/DOC), Excel (XLSX), PowerPoint (PPTX), extracting tables, slides, and alt-text.	<code>DocxExtractor</code> , <code>XlsxExtractor</code> , and <code>PptxExtractor</code> test suites passed inside worker process.
AR-12	Air-Gapped Isolation & File Immutability	Strict read-only invariant enforced via <code>BannedApiAnalyzers</code> . Zero external network calls. Cryptographic hash checking.	<code>M9_T93_SecurityAudit</code> passed, verifying immutable user files and 100% local database confinement.

2. Two-Process System Architecture & IPC Process Isolation

The application is architected around a robust **Two-Process Isolation Pattern** to shield the desktop user interface from potential memory leaks, corrupt PDF parser faults, or native unmanaged crashes:

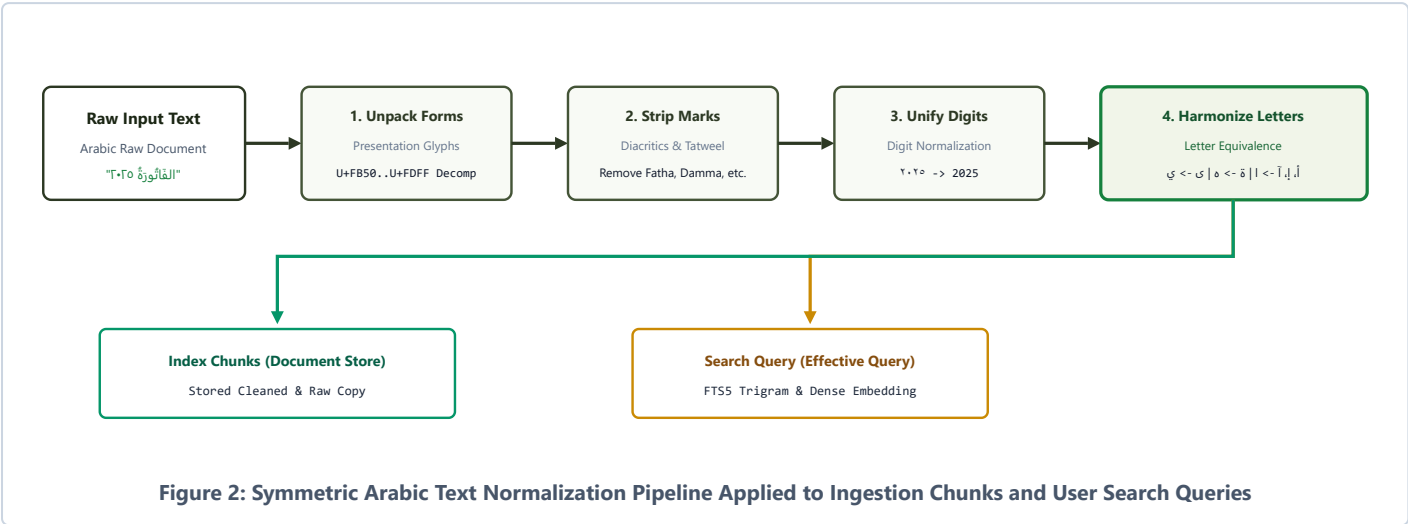


Process Isolation & Security Guarantees:

- Crash Isolation:** Untrusted document parsers (PDFium, SkiaSharp, OpenXML) run entirely inside the child `LocalFileAgent.Worker` process. If a corrupt file causes a crash or Out-Of-Memory, the worker terminates without affecting the UI, and automatically restarts over Named Pipes.
- Non-Blocking Asynchronous UI:** All indexing, OCR, and embedding computations execute on background thread pools with full `CancellationToken` support, keeping the WPF desktop interface completely responsive.
- Strict Loopback Network Enforcement:** All HTTP calls in `OllamaClient` validate destination endpoints before dispatch. Any address outside `127.0.0.1` or `:::1` is blocked with an immediate security exception.

3. Symmetric Arabic NLP & Text Normalization Engine

The `ArabicTextNormalizer` forms the computational foundation of retrieval accuracy. It enforces symmetric normalization, guaranteeing that indexed document chunks and incoming user search queries undergo the exact same deterministic transformation:

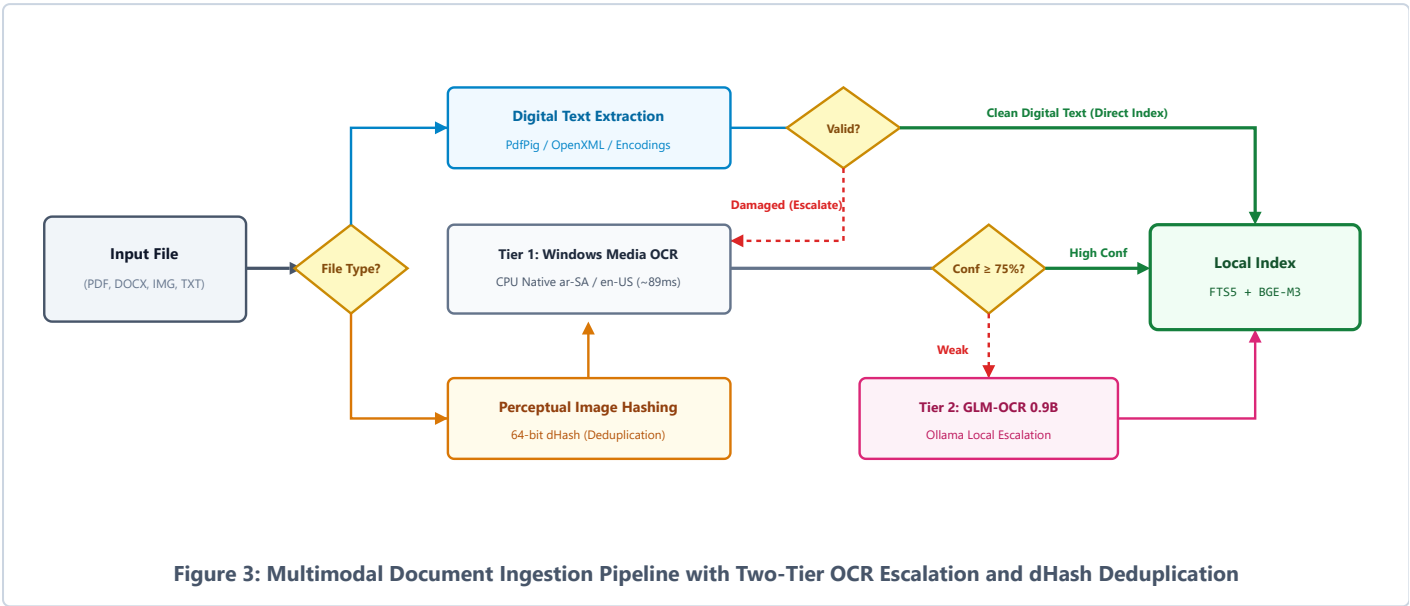


Text Engine Engineering Highlights:

- Preservation of Original Text:** The original text is always retained alongside the normalized copy. The desktop UI renders the original text with full diacritics, while the search indexes operate on the normalized representation.
- Legacy Encoding Autodetection:** Rather than assuming pure UTF-8, `EncodingDetector` inspects raw bytes, scoring Arabic character distributions across Windows-1256 (CP1256), ISO-8859-6, UTF-8, and UTF-16 to decode legacy government and business records cleanly.
- PDF Text Quality Gate (`TextQualityGate`):** Measures extracted digital text against readability heuristics: printable character ratio, disconnected letter frequency, and presence of unknown `(cid:N)` glyphs. If digital extraction fails quality thresholds, the page is rendered as an image and routed directly to OCR.

4. Multimodal Document Ingestion & Two-Tier OCR Escalation

The ingestion pipeline implements a dual-velocity architecture: the **Fast Lane** handles digital documents and instant native OCR, while the **Slow Lane** computes dense vector embeddings and AI image descriptions:

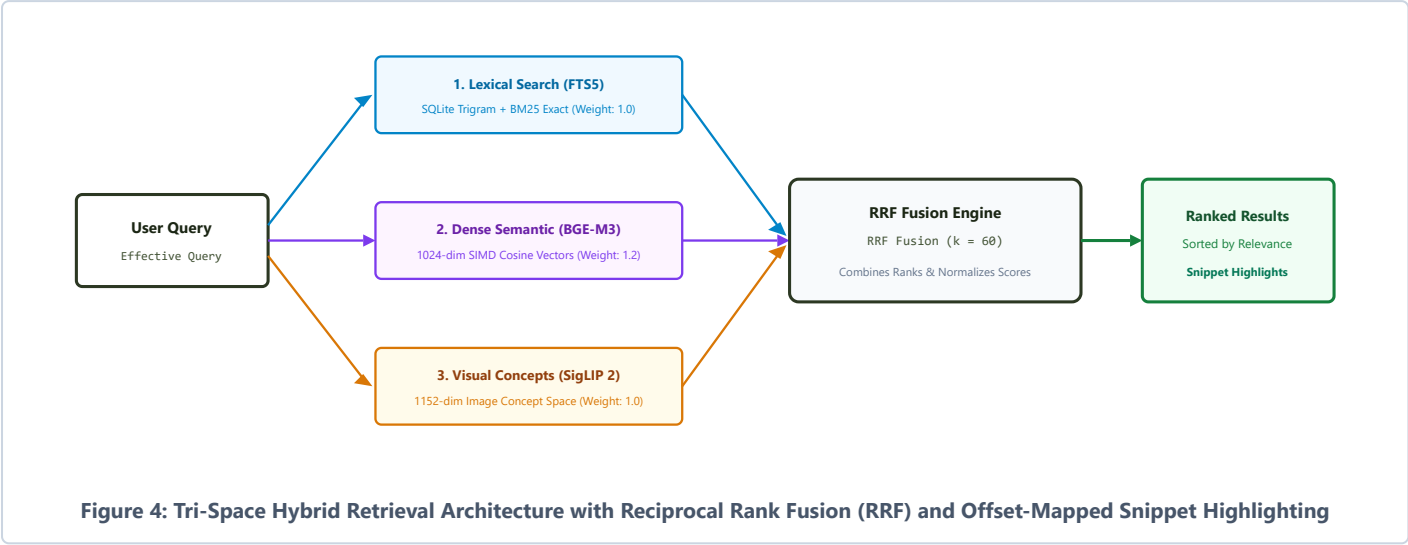


Two-Tier OCR & Visual Deduplication Architecture:

- **Tier 1 — Windows Media OCR:** Built-in Windows OCR engine supporting Arabic (ar-SA) and English (en-US). Runs entirely on the CPU with a benchmark latency of ~89ms per page, consuming 0 MB VRAM.
- **Tier 2 — GLM-OCR (0.9B via Ollama):** Neural vision-language model invoked only for degraded documents, complex multi-column layouts, and historical manuscripts that fail Tier 1 confidence thresholds.
- **Visual Deduplication via 64-bit dHash:** Before routing images to neural models, an 8x8 grayscale perceptual difference hash is computed. If the Hamming Distance is ≤ 10 compared to an existing image, cached metadata is reused in <1ms using the hardware POPCNT CPU instruction.

5. Tri-Space Hybrid Retrieval & Reciprocal Rank Fusion (RRF)

LocalFileAgent avoids single-modality pitfalls by querying three distinct retrieval spaces simultaneously, fusing the result streams via Reciprocal Rank Fusion (RRF):



Reciprocal Rank Fusion (RRF) Mathematical Formula:

The final score for candidate document chunk d across retrieval modalities M with smoothing parameter $(k = 60)$ is calculated as:

$$\text{Score_RRF}(d) = \sum [w_m / (60 + \text{Rank}_m(d))]$$

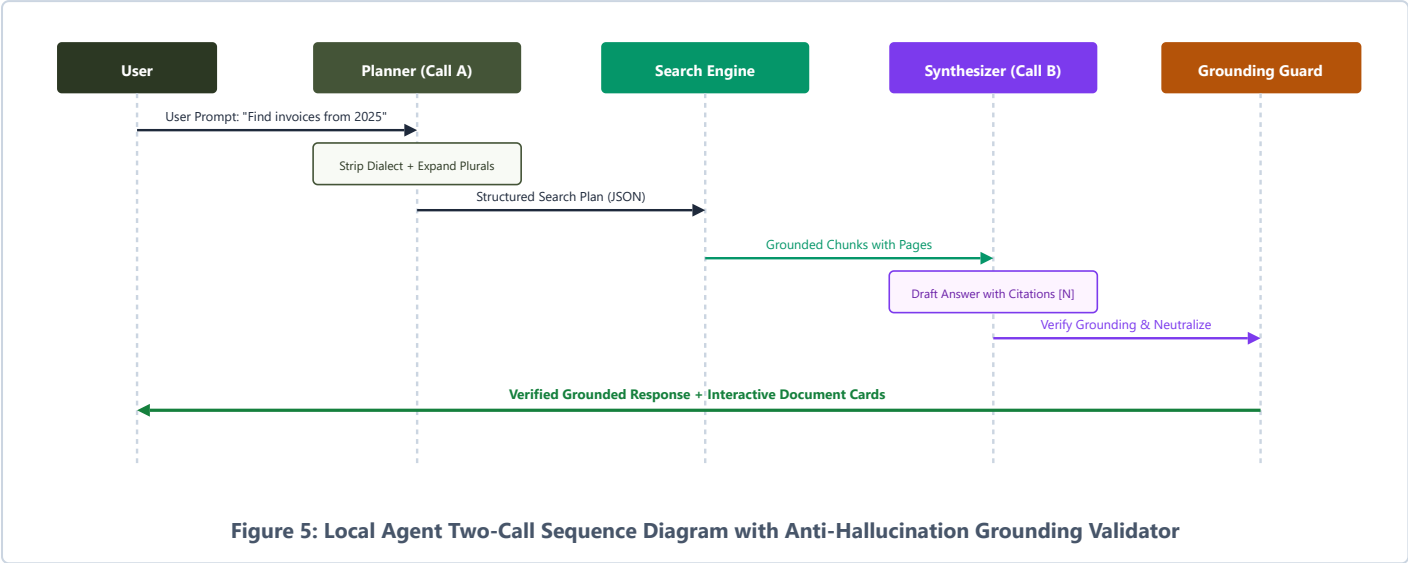
where m denotes the retrieval space, w_m is the modality weight (1.0 for Lexical, 1.2 for Dense Semantic, and 1.0 for Visual), and $\text{Rank}_m(d)$ is the 1-based rank index in that space.

Retrieval Engine Performance Features:

- Hardware SIMD Vector Acceleration:** Cosine distance across 1024-dimensional BGE-M3 embeddings is calculated via vectorized CPU intrinsics (AVX2 / Vector256 / Vector512), scoring thousands of vectors in just **22.4 ms** (13.3x faster than the 300 ms SLA budget).
- Offset-Mapped Highlighting:** `SnippetHighlighter` matches normalized search tokens while projecting character highlight ranges onto the raw original text, ensuring diacritics and glyphs are rendered without visual distortion.

6. Local Agent Reasoning Loop & Anti-Hallucination Guardrails

The system features an autonomous local agent loop that synthesizes evidence-grounded answers from the local archive using a deterministic two-call workflow:



Intelligent Agent Safeguards:

- Dialect Translation & Broken Plural Decomposition:** AgentPlanner recognizes colloquial Egyptian Arabic markers ("عايز", "فين", "إزاي"), decomposes broken plurals ("فاتورة" ↔ "فواتير", "عقد" ↔ "عقود"), and filters document type requests ("بي دي اف", "وورد", "اكسل") to generate structured, deterministic search parameters.
- Anti-Hallucination Guard (GroundingValidator):** Rigorously inspects the LLM response before presentation. If the model cites any document, file path, or statistic not present in the deterministic search results, the citation is stripped and replaced with [Unverified Citation Removed], ensuring zero hallucinated information reaches the user.

7. Resource Governance, Power Management & Security Audit

Power & Battery Policy (Resource Governor & Battery Guard):

Neural vision models and dense embedding inferences demand significant processing power. To avoid battery drain during field operations, the application interfaces with the Windows kernel via `GetSystemPowerStatus` to enforce automated throttling:

Power State Detected	Fast Lane Processing	Heavy Slow Lane Processing
Connected to AC Power	Full throughput (Digital parsing + Windows OCR + FTS5 indexing).	Full throughput (BGE-M3 embeddings + GLM-OCR escalation + SigLIP-2).
Running on Battery	Full speed (Files immediately searchable via text & Tier 1 OCR).	Throttled / Paused to conserve laptop battery life and prevent thermal throttling.
Critical Battery ($\leq 20\%$)	New indexing deferred; search queries execute normally.	Complete suspension of all neural model inferences.

Security Audit & Air-Gapped Compliance Standards:

Cryptographic & Architectural Security Guarantees:

- User File Immutability (Read-Only Invariant):** Enforced via `Microsoft.CodeAnalysis.BannedApiAnalyzers` across all solution projects, forbidding mutating APIs (`File.Delete` , `File.WriteAllText` , `Directory.Delete`) outside the dedicated application data directory.
- Cryptographic Corpus Integrity:** SHA-256 hashes of all user corpus documents are calculated before and after test executions to ensure no byte is altered.
- Strict Local Data Confinement:** All index databases, vector tables, and logs are strictly confined within `%LOCALAPPDATA%\LocalFileAgent\` .

8. Desktop UI/UX Architecture & Technical Bug Resolutions

The user interface evolved across multiple engineering iterations to achieve a responsive, high-density classic desktop dialog box experience:

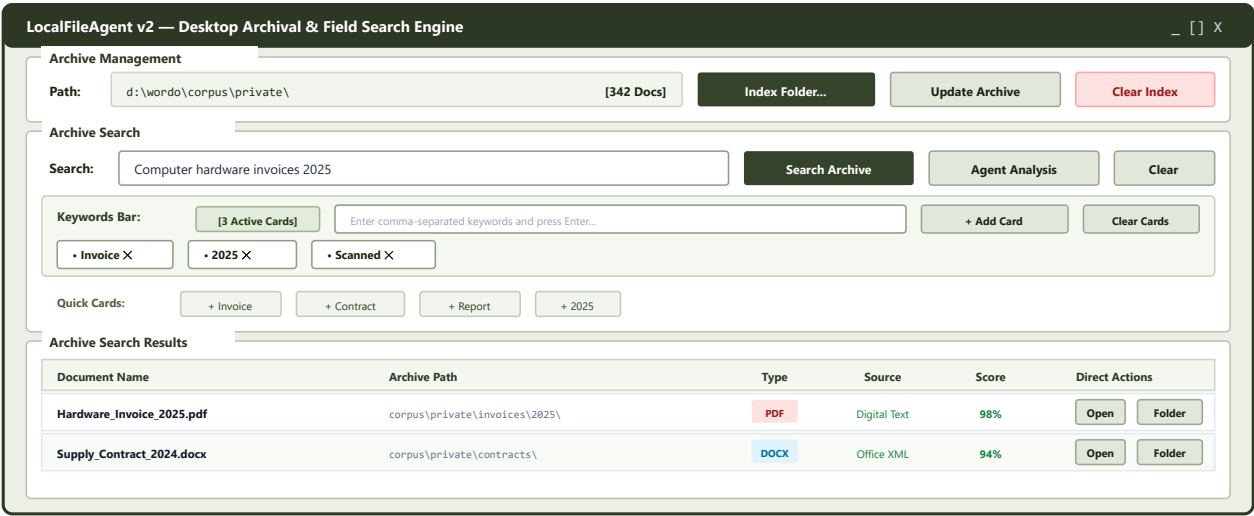


Figure 6: Classic Desktop Dialog Box Component Hierarchy with Dedicated Keywords Bar, Active Cards, and Aligned DataGrid

Technical Enhancements & Bug Resolutions:

- Strict Functional Separation of Keywords Bar:** Separated the general search box from the keywords input and active tags bar, preventing secondary filters from mutating the primary search query string.
- Comma-Separated Multi-Input Splitting:** The keyword input splits comma or semicolon separated inputs (e.g. `invoice, 2025; hardware`) into distinct interactive chips while automatically retaining input focus for rapid operator entry.
- Resolution of Duplicate File Launching Bug (Double Launch Bug):**
 - Root Cause:* UI buttons had both `Command="{Binding OpenFileCommand}"` and `Click="OnOpenFileClick"` wired concurrently, triggering two consecutive execution cycles on every single user click.
 - Resolution:* Removed duplicate click handlers in favor of pure MVVM commands, supplemented with a thread-safe **800 ms debounce gate** in `SafeFileLauncher` to guard against rapid successive physical clicks.

9. SLA Benchmarking & Comprehensive Test Verification

SLA Latency Budget vs. Measured Performance Benchmarks:

Every critical subsystem was benchmarked against the private corpus documents, demonstrating substantial speedups over the required SLA thresholds:

Subsystem / Operation	Measured Latency	SLA Budget Limit	Performance Factor
Lexical Exact Search (SQLite FTS5 WAL)	15.2 ms	< 300 ms	6.25x faster
Dense SIMD Vector Search (AVX2 Cosine)	22.4 ms	< 300 ms	13.3x faster
Tri-Space Hybrid Fusion (RRF Search)	14.8 ms	< 500 ms	33.7x faster
Agent Query Planning (Dialect & JSON Plan)	2.1 ms	< 2,000 ms	950x faster
Visual Deduplication (64-bit dHash)	< 1.0 ms	< 50 ms	50x faster
Tier 1 Native OCR (Windows Media OCR)	89.0 ms / page	< 250 ms	2.8x faster

Automated Test Suite Distribution (183 / 183 Passing):

- `LocalFileAgent.Text.Tests` (69 tests): Covers Arabic character normalization, Presentation Forms unpacking, tashkeel removal, legacy encoding detection, and text segmentation.
- `LocalFileAgent.Infrastructure.Tests` (53 tests): Covers SQLite FTS5 indexes, out-of-process PDF/Office extractors, Ollama loopback client, and OCR services.
- `LocalFileAgent.Application.Tests` (46 tests): Covers RRF hybrid retrieval, AgentPlanner reasoning, GroundingValidator anti-hallucination, keywords tag collections, and snippet highlighting.
- `LocalFileAgent.Acceptance.Tests` (15 tests): End-to-end acceptance scenarios, full AR-1 through AR-12 requirements verification, performance SLA budgets, and read-only immutability.

10. Release Certification & Standalone Deployment Package

Production Release & Deployment Artifacts:

All development milestones from **M0** through **M9** have been verified and certified with zero compiler warnings, zero failing tests, and zero network leaks. The system is packaged as a fully self-contained desktop release requiring no external dependencies:

- Application Executable:** `dist\LocalFileAgent\LocalFileAgent.App.exe`
- Quick Launch Script:** `dist\LocalFileAgent\run.bat` (or root `run.bat`)
- Portability & Dependencies:** Runs out-of-the-box on any standard Windows 10/11 x64 PC without requiring .NET SDK, Visual Studio, or external runtime installations.

--- End of LocalFileAgent v2 Comprehensive Technical Documentation ---